Scaffold en Jetpack Compose

Nota

- En Compose se llama `Scaffold` (a veces se escribe por error “scafold”).

Que es `Scaffold`

- Es un composable de Material que proporciona la **estructura base** de una pantalla/app.
- Te permite colocar elementos “de marco” sin repetir codigo:
  - `topBar` (barra superior)
  - `bottomBar` (barra inferior)
  - `floatingActionButton`
  - `snackbarHost`
  - `content` (contenido principal con `padding` interno)

Cuando usarlo

- En la raiz de tu UI (nivel de app) para que todas las pantallas compartan el mismo marco.
- En pantallas concretas si solo algunas necesitan una estructura especial.

Ejemplo minimo

```
@Composable
fun PantallaBasica() {
    Scaffold(
        topBar = { TopAppBar(title = { Text("Titulo") }) }
    ) { innerPadding ->
        Column(modifier = Modifier.padding(innerPadding)) {
            Text("Contenido")
        }
    }
}
```

Importante: usar `innerPadding`

- El `content` de `Scaffold` recibe un `PaddingValues` para evitar que tu contenido quede debajo del `TopAppBar` o del `BottomBar`.
- Norma didactica: **siempre** aplicar `Modifier.padding(innerPadding)` en el contenedor principal.

Mostrar u ocultar el TopBar segun la pagina actual

Idea

- El `NavHost` decide que pantalla dibujar.
- `Scaffold` puede decidir si muestra el `topBar` consultando la ruta actual.
- Las paginas no deben recibir `NavController` ; mejor callbacks/eventos.

Ejemplo (NavGraph + Scaffold + TopBar condicional)

- En este ejemplo:
  - NO mostramos TopBar en `login` .
  - Mostramos TopBar en el resto.
  - En `detalle/{id}` mostramos boton de volver.

```
object Rutas {
    const val LOGIN = "login"
    const val LISTA = "lista"
    const val DETALLE = "detalle/{id}"
    fun detalle(id: String) = "detalle/$id"
}

@Composable
fun AppRoot() {
    val navController = rememberNavController()

    // Observa el destino actual
    val backStackEntry by navController.currentBackStackEntryAsState()
    val currentRoute = backStackEntry?.destination?.route

    // Si no coincide exactamente (p.e. por parametros), usa startsWith
    val isLogin = currentRoute == Rutas.LOGIN
    val isDetalle = currentRoute?.startsWith("detalle/") == true || currentRoute == Rutas.DETALI

    Scaffold(
        topBar = {
            if (!isLogin) {
                TopAppBar(
                    title = {
                        Text(
                            when {
                                isDetalle -> "Detalle"
                                currentRoute == Rutas.LISTA -> "Lista"
                                else -> "App"
                            }
                        )
                    },
                    navigationIcon = {
                        if (isDetalle) {
                            IconButton(onClick = { navController.popBackStack() }) {
                                Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = '
                            }
                        }
                    }
                )
            }
        }
    ) { innerPadding ->
```

```
        NavHost(
            navController = navController,
            startDestination = Rutas.LOGIN,
            modifier = Modifier.padding(innerPadding)
        ) {
            composable(Rutas.LOGIN) {
                LoginPagina(
                    onLoginOk = { navController.navigate(Rutas.LISTA) {
                        popUpTo(Rutas.LOGIN) { inclusive = true }
                    } }
                )
            }
            composable(Rutas.LISTA) {
                ListaClasePagina(
                    onAlumnoClick = { id -> navController.navigate(Rutas.detalle(id)) }
                )
            }
            composable(Rutas.DETALLE) { backStack ->
                val id = backStack.arguments?.getString("id") ?: ""
                DetalleAlumnoPagina(id = id)
            }
        }
    }
}
```

Comentarios didacticos

- La condicion del TopBar vive en el nivel “root” (app), no dentro de cada pagina.
- `currentRoute` puede venir con rutas parametrizadas; por eso a veces conviene comprobar con `startsWith` .
- Si tu app crece, puedes extraer la logica a una funcion:
  - `fun shouldShowTopBar(route: String?): Boolean`

Variantes habituales

- Ocultar tambien `bottomBar` en pantallas concretas (login, splash, onboarding).
- Cambiar acciones del TopBar segun destino (iconos, menu).
- Gestionar `SnackbarHostState` desde `Scaffold` para mensajes globales.

Checklist rapido

- `Scaffold` en el nivel correcto (app/pantalla).
- `Modifier.padding(innerPadding)` aplicado.

- TopBar condicional basado en ruta/destino.
- Paginas con callbacks, sin `NavController`.